
Software Design Document (SDD)

Module: fruit_db Driver / Data Module

Version: 1.1

Authors: Steven McClellan, Vania Leal, Leonardo García, Melanie Picen

1. Introduction

1.1. Purpose

This document describes the design and architecture of the `fruit_db` module. The module implements a small static fruit database and a geometric color-classification mechanism based on normalized RGB proportions. It provides details on data structures, algorithms, control flow, public APIs, and internal helper functions to guide implementation, maintenance, and testing.

1.2. Scope

The `fruit_db` module is a lightweight embedded software component intended to store calibrated fruit color signatures and compare current sensor readings against those references. It does not access hardware registers directly. Instead, it receives raw or normalized RGB values from a higher-level sensor/application module and converts them into chromatic proportions for classification.

The module supports:

- Initialization of a fixed-size fruit database.
- Loading factory default fruit references.
- Adding calibrated fruit records.
- Deleting fruit records by index.
- Returning the number of valid fruit records.
- Returning a read-only pointer to the internal table.
- Comparing a current RGB reading against a selected database entry using Euclidean distance in normalized RGB space.

1.3. Definitions, Acronyms, and Abbreviations

Term	Definition
RGB	Red, Green, Blue color components.
Chromatic proportion	A normalized color value calculated as one RGB component divided by the sum of all RGB components.

Term	Definition
Euclidean distance	Geometric distance between two points. In this module, it is used to compare measured and reference RGB proportions.
Calibration	Process of saving a measured fruit color as a reference in the database.
Database slot	One position inside the fixed-size fruit record array.
Valid record	A database entry marked as occupied through the <code>es_valida</code> flag.
API	Application Programming Interface. Public functions exposed through <code>fruit_db.h</code> .
SDD	Software Design Document.
FR	Functional Requirement.
NFR	Non-functional Requirement.

1.4. Requirements

A separate SRS file was not provided for this module. Therefore, the requirements below are derived from the `fruit_db.h` public API and the `fruit_db.c` implementation.

Functional Requirements

Req. ID	Function	Requirement Statement
FR-1	<code>fruit_db_init()</code>	The system shall initialize the internal fruit database and load factory default fruit references.
FR-2	<code>fruit_db_add()</code>	The system shall add a new fruit record using a name and RGB readings normalized into chromatic proportions.
FR-3	<code>fruit_db_delete()</code>	The system shall delete a fruit record by index and shift following records to avoid empty gaps.
FR-4	<code>fruit_db_compare()</code>	The system shall compare a current RGB reading against a selected reference and return whether the match is acceptable.
FR-5	<code>fruit_db_get_count()</code>	The system shall return the current number of valid fruit records.
FR-6	<code>fruit_db_get_table()</code>	The system shall return a read-only pointer to the internal fruit database table.

Non-functional Requirements

- NFR-1: The module shall use static memory only. No dynamic allocation is used.
- NFR-2: The module shall limit the database to `DB_MAX_FRUITS` records.
- NFR-3: The module shall limit fruit names to `DB_MAX_NAME_LEN - 1` visible characters plus the null terminator.

- NFR-4: The module shall avoid division by zero during RGB normalization by substituting a sum of 1.0f when the RGB sum is zero.
 - NFR-5: The module shall use `sqrtf()` from `<math.h>` to compute Euclidean distance for color comparison.
 - NFR-6: The module shall expose database content as read-only through `const fruta_registro_t*`.
-

2. System Architecture

2.1. High-Level Design

The `fruit_db` module follows a layered architecture, but it is not a direct hardware driver. It acts as an application-level data and classification module.

Layer	Files / Modules	Description
Application Layer	<code>main.c</code> , menu logic, user interface logic	Calls the <code>fruit_db</code> API to initialize, add, delete, display, and compare fruits.
Sensor / Acquisition Layer	Color sensor module, ADC module, or RGB acquisition logic	Produces raw RGB values used by <code>fruit_db_add()</code> and <code>fruit_db_compare()</code> .
<code>fruit_db</code> Module	<code>fruit_db.c</code> / <code>fruit_db.h</code>	Stores fruit records, normalizes RGB readings, compares color signatures, and returns status codes.
Standard C Library Layer	<code><stdint.h></code> , <code><math.h></code>	Provides fixed-width integer types and <code>sqrtf()</code> for distance calculation.

2.2. Module Interfaces

- **Public API:** Functions and data types declared in `fruit_db.h`.
 - **Implementation:** Static database storage and algorithm logic implemented in `fruit_db.c`.
 - **Dependencies:**
 - `<stdint.h>` for `uint8_t`.
 - `<math.h>` for `sqrtf()`.
 - **No direct hardware dependency:** The module does not configure GPIO, ADC, timers, UART, or other MCU registers.
-

3. Detailed Design

3.1. Class Diagram

The following diagram represents the static structure of the `fruit_db` module and its relationship with the application and sensor layers.

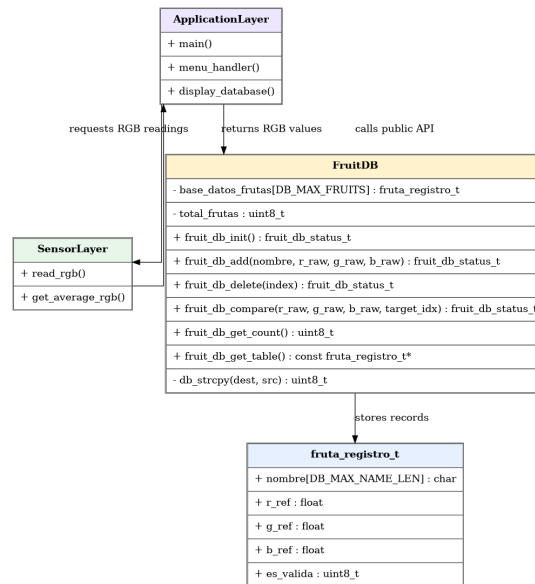


Figure 1. Class Diagram - `fruit_db` Module

3.2. Module Behavior

`fruit_db_init()` - Database Initialization (FR-1)

`fruit_db_init()` clears all database slots by setting `es_valida = 0`. Then it loads two factory fruit references:

- Manzana Roja: (`r_ref = 0.55f`, `g_ref = 0.25f`, `b_ref = 0.20f`)
- Limon: (`r_ref = 0.30f`, `g_ref = 0.55f`, `b_ref = 0.15f`)

Each loaded fruit increments `total_frutas`.

Sequence Diagram — fruit_db_init

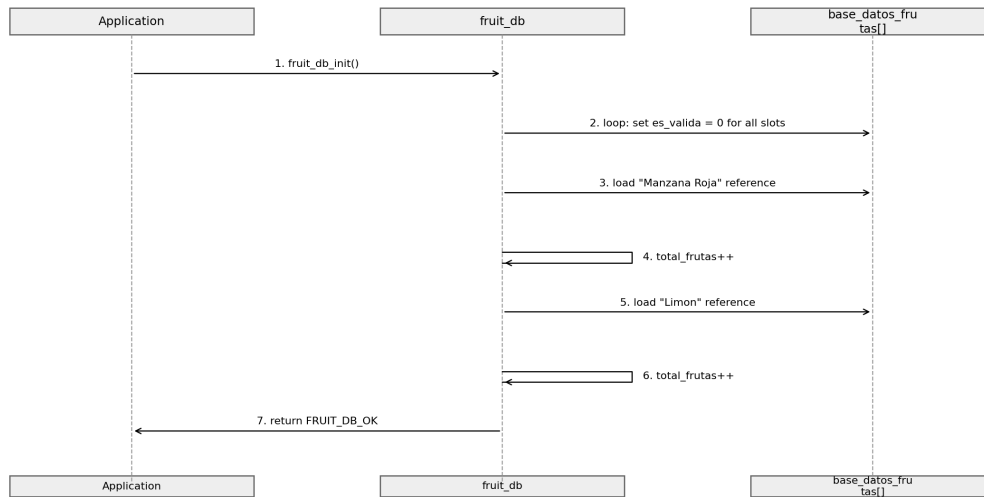


Figure 2. Sequence Diagram - fruit_db_init

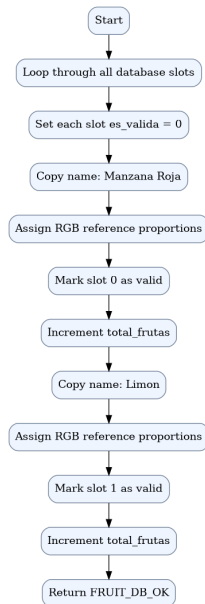


Figure 3. Activity Diagram - fruit_db_init

fruit_db_add() - Add Calibrated Fruit (FR-2)

fruit_db_add() searches for the first invalid slot. When an empty slot is found, it computes the sum of the input RGB values and normalizes each component into a proportion:

```

suma = r_raw + g_raw + b_raw
r_ref = r_raw / suma
g_ref = g_raw / suma
b_ref = b_raw / suma

```

If `suma == 0.0f`, it is changed to `1.0f` to avoid division by zero. The fruit name is copied using the internal bounded copy function `db_strcpy()`. The record is marked valid, `total_frutas` is

incremented, and FRUIT_DB_OK is returned. If no empty slot exists, the function returns FRUIT_DB_ERR_FULL.

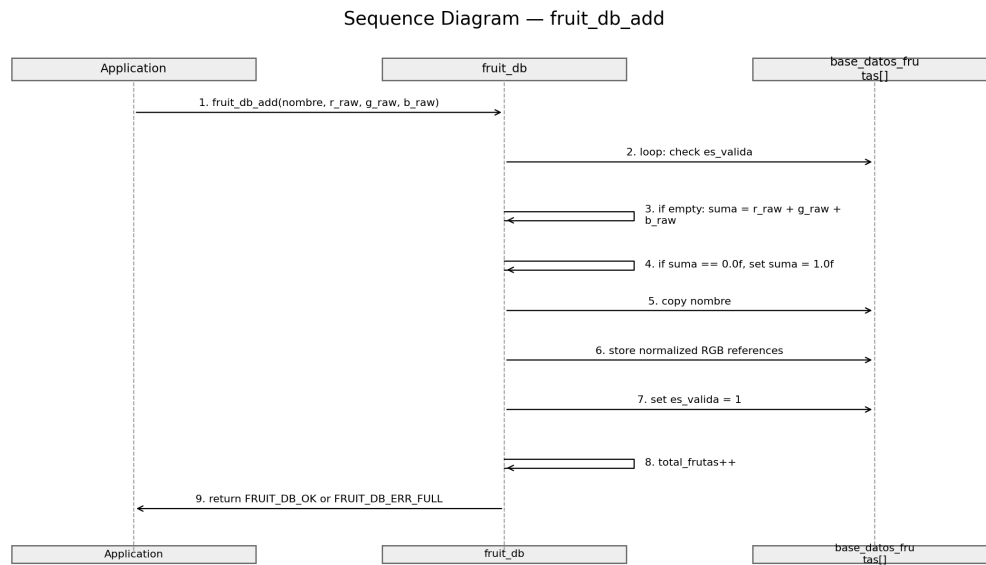


Figure 4. Sequence Diagram - fruit_db_add

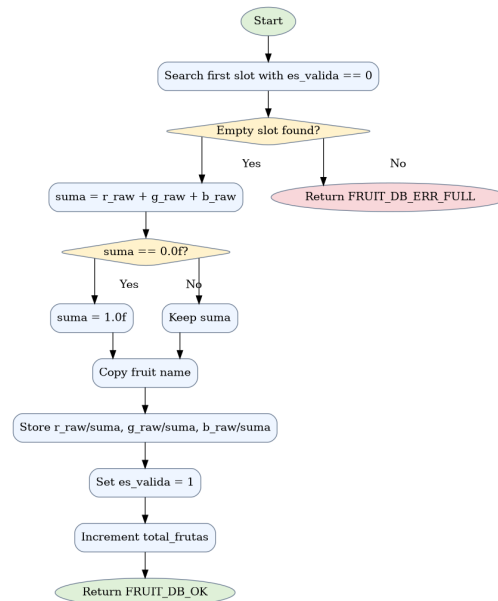


Figure 5. Activity Diagram - fruit_db_add

fruit_db_delete() - Delete Fruit Record (FR-3)

fruit_db_delete() validates that the requested index is lower than total_frutas. If the index is invalid, it returns FRUIT_DB_ERR_INVALID_INDEX. If valid, the function shifts all following records one position to the left, clears the last duplicated slot by setting es_valida = 0, decrements total_frutas, and returns FRUIT_DB_OK.

Sequence Diagram — fruit_db_delete

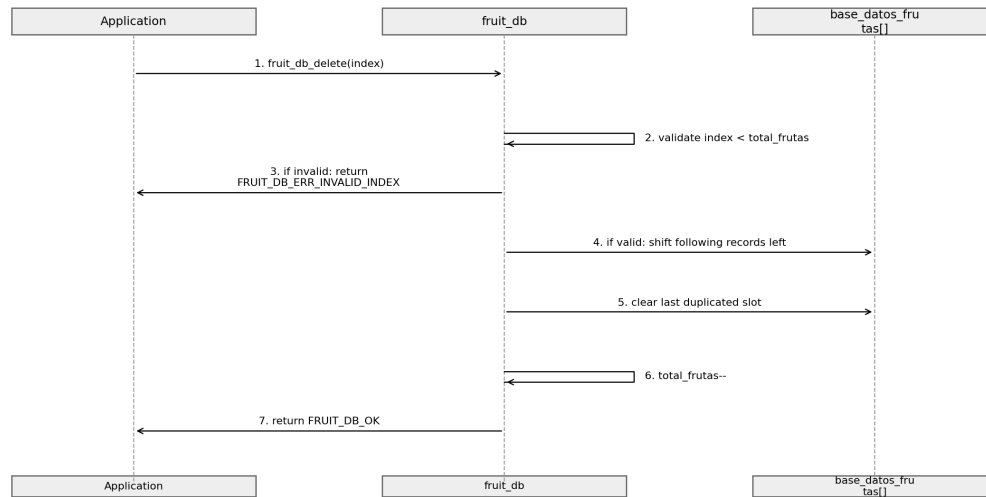


Figure 6. Sequence Diagram - fruit_db_delete

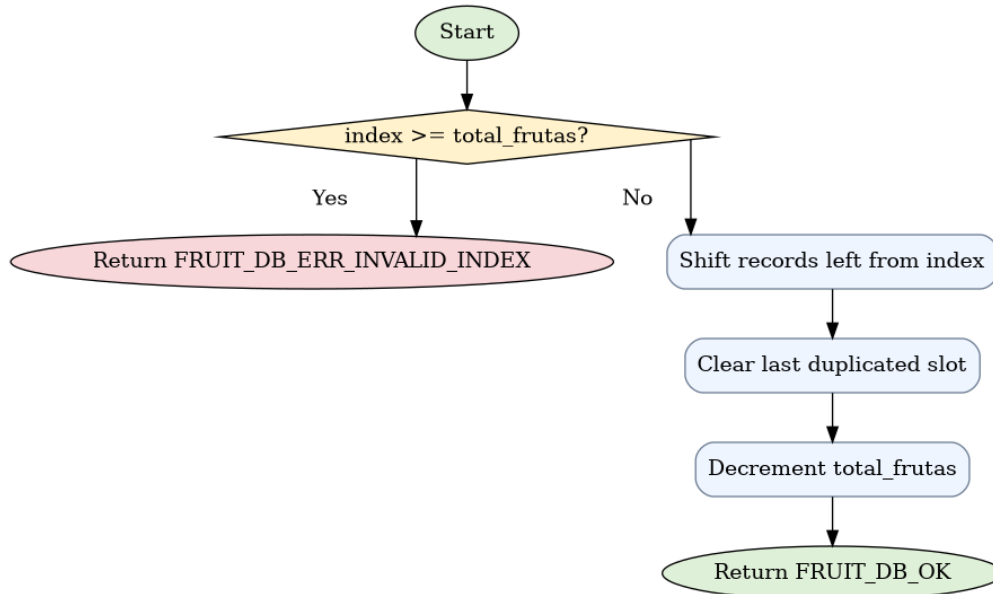


Figure 7. Activity Diagram - fruit_db_delete

fruit_db_compare() - Color Matching (FR-4)

fruit_db_compare() validates that target_idx is within DB_MAX_FRUITS and that the selected record is valid. If the selected record is not available, it returns FRUIT_DB_ERR_INVALID_INDEX.

For a valid target, the function normalizes the current RGB reading into chromatic proportions. It then calculates the Euclidean distance between the measured proportions and the stored reference:

```

dr = r_med - r_ref
dg = g_med - g_ref
db = b_med - b_ref

```

```
distancia = sqrtf((dr * dr) + (dg * dg) + (db * db))
```

If `distancia <= DB_MATCH_TOLERANCE`, the reading is accepted as a match and the function returns `FRUIT_DB_OK`. Otherwise, it returns `FRUIT_DB_ERR_NO_MATCH`.

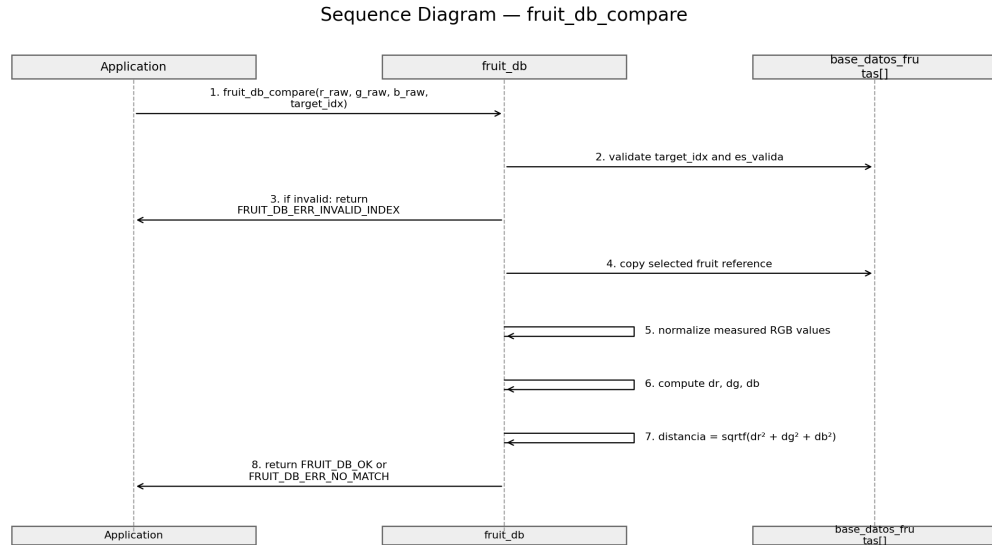


Figure 8. Sequence Diagram - `fruit_db_compare`

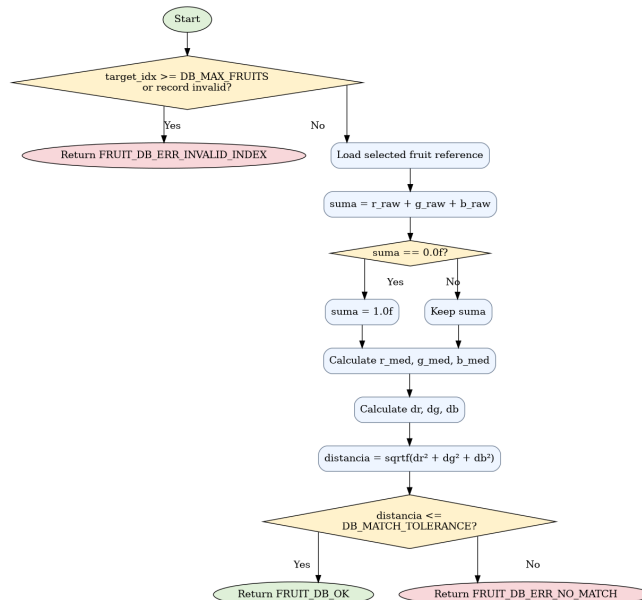


Figure 9. Activity Diagram - `fruit_db_compare`

Driver State Machine

The module behaves like a small database state machine. After reset, the static variables exist but the database content should not be considered ready until `fruit_db_init()` is called.

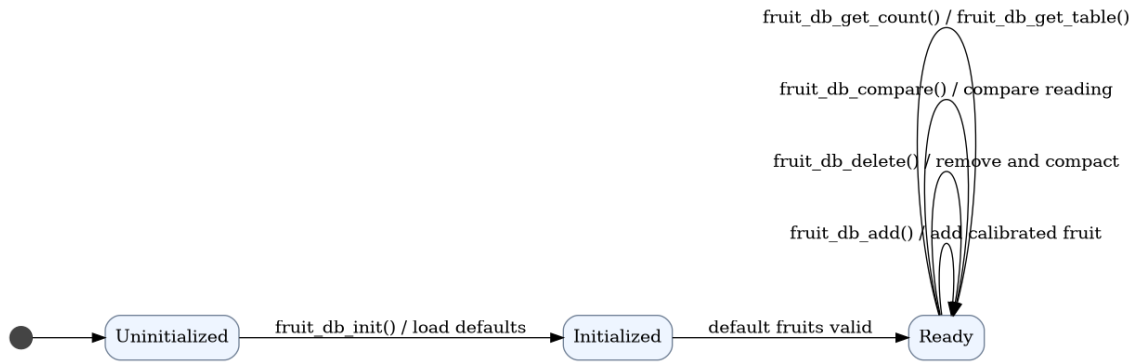


Figure 10. State Diagram - fruit_db Lifecycle

3.3. fruit_db Driver / Data Module

The fruit_db module stores a fixed number of fruit color references in the static array `base_datos_frutas`. Each reference uses the `fruta_registro_t` structure, which includes a name, normalized red/green/blue proportions, and a validity flag.

The classification strategy is based on chromaticity instead of absolute RGB magnitude. This means the module divides each color component by the total RGB sum. For example, if a sensor reading is brighter or dimmer but has similar relative color distribution, the normalized values remain close. This helps reduce the effect of lighting intensity changes.

The comparison logic treats each fruit color as a point in 3D normalized RGB space:

```

Fruit reference: (r_ref, g_ref, b_ref)
Measured reading: (r_med, g_med, b_med)
  
```

The distance between the two points is calculated using Euclidean distance. A fruit is considered a match only if the distance is less than or equal to `DB_MATCH_TOLERANCE`, currently defined as `0.15f`.

4. Application Programming Interfaces

4.1. Data Types and Macros

Macros

Macro Name	Value	Notes / Definition
<code>DB_MAX_NAME_LEN</code>	<code>20U</code>	Maximum number of characters reserved for the fruit name, including the null terminator. The visible name length is limited to 19 characters.
<code>DB_MAX_FRUITS</code>	<code>10U</code>	Maximum capacity of the internal fruit database.
<code>DB_MATCH_TOLERANCE</code>	<code>0.15f</code>	

Macro Name	Value	Notes / Definition
		Maximum Euclidean color distance accepted as a match.

fruta_registro_t

fruta_registro_t stores one fruit color signature.

Member Name	Type	Notes / Definition
nombre	char[DB_MAX_NAME_LEN]	Fruit name string. Copied with db_strcpy() and always null-terminated.
r_ref	float	Normalized red proportion reference.
g_ref	float	Normalized green proportion reference.
b_ref	float	Normalized blue proportion reference.
es_valida	uint8_t	Record state flag. 1 means occupied/valid, 0 means empty/invalid.

fruit_db_status_t

fruit_db_status_t is an enumeration used by the API to represent operation result codes.

Enumerator	Value	Notes / Definition
FRUIT_DB_OK	0	Operation completed successfully or comparison matched.
FRUIT_DB_ERR_FULL	1	No empty slot is available in the database.
FRUIT_DB_ERR_INVALID_INDEX	2	The selected index is outside the valid range or points to an empty record.
FRUIT_DB_ERR_NO_MATCH	3	The measured color is outside the match tolerance for the selected fruit.

4.2. Error Codes

The fruit_db API is intended to use fruit_db_status_t as its status code type. The error codes relevant to this module are listed below.

Code	Notes / Definition
FRUIT_DB_OK	0x00 - Operation was successful. For fruit_db_compare(), it means the measured fruit color matched the selected reference.
FRUIT_DB_ERR_FULL	0x01 - The database is full and a new fruit cannot be added.
FRUIT_DB_ERR_INVALID_INDEX	0x02 - The requested index is invalid or the selected database slot is not valid.
FRUIT_DB_ERR_NO_MATCH	

Code	Notes / Definition
	0x03 - The measured RGB proportions do not match the selected reference within DB_MATCH_TOLERANCE.

4.3. Function Definitions

fruit_db_init

Function Name	fruit_db_init	
Syntax	fruit_db_status_t fruit_db_init(void)	Declared in fruit_db.h. The current .c implementation uses uint8_t fruit_db_init(void), which is compatible in value usage but should be corrected for type consistency.
Parameters (in)	None	No input parameters.
Parameters (inout)	None	No inout parameters.
Parameters (out)	None	No output parameters.
Return value	fruit_db_status_t	Returns FRUIT_DB_OK after clearing the table and loading default fruit references.
Description	Initializes the internal database table by marking all slots invalid, then loads factory entries for Manzana Roja and Limon.	
Constraints	In the current implementation, total_frutas is not reset to 0 before loading defaults. Calling this function more than once can produce an incorrect count.	
Available via	fruit_db.h	

fruit_db_add

Function Name	fruit_db_add	
Syntax	fruit_db_status_t fruit_db_add(const char *nombre, float r_raw, float g_raw, float b_raw)	Declared in fruit_db.h. The current .c implementation uses uint8_t fruit_db_add(...), which should be corrected for type consistency.
Parameters (in)	nombre	Pointer to a null-terminated string containing the fruit name.
Parameters (in)	r_raw	Raw or averaged red sensor value.
Parameters (in)	g_raw	

		Raw or averaged green sensor value.
Parameters (in)	b_raw	Raw or averaged blue sensor value.
Parameters (inout)	None	No inout parameters.
Parameters (out)	None	No output parameters.
Return value	fruit_db_status_t	Returns FRUIT_DB_OK if the fruit is added. Returns FRUIT_DB_ERR_FULL if no slot is available.
Description	Finds the first empty database slot, normalizes the RGB input values, copies the fruit name, stores the reference proportions, marks the slot valid, and increments total_frutas.	
Constraints	Does not validate whether nombre is NULL. If nombre == NULL, the internal copy function may dereference an invalid pointer. Does not reject negative RGB values.	
Available via	fruit_db.h	

fruit_db_delete

Function Name	fruit_db_delete	
Syntax	fruit_db_status_t fruit_db_delete(uint8_t index)	Declared in fruit_db.h. The current .c implementation uses uint8_t fruit_db_delete(uint8_t index), which should be corrected for type consistency.
Parameters (in)	index	Index of the fruit record to delete. Valid range is 0 to total_frutas - 1.
Parameters (inout)	None	No inout parameters.
Parameters (out)	None	No output parameters.
Return value	fruit_db_status_t	Returns FRUIT_DB_OK if deletion succeeds. Returns FRUIT_DB_ERR_INVALID_INDEX if the index is outside the current valid count.
Description	Removes the selected fruit by shifting all following records one position to the left, then clears the	

	last duplicated slot and decrements the record count.	
Constraints	The function assumes valid records are compacted from index 0 to <code>total_frutas - 1</code> . This is maintained by the delete algorithm.	
Available via	<code>fruit_db.h</code>	

fruit_db_compare

Function Name	<code>fruit_db_compare</code>	
Syntax	<code>fruit_db_status_t fruit_db_compare(float r_raw, float g_raw, float b_raw, uint8_t target_idx)</code>	Declared in <code>fruit_db.h</code> . The current .c implementation uses <code>uint8_t fruit_db_compare(...)</code> , which should be corrected for type consistency.
Parameters (in)	<code>r_raw</code>	Current red sensor reading.
Parameters (in)	<code>g_raw</code>	Current green sensor reading.
Parameters (in)	<code>b_raw</code>	Current blue sensor reading.
Parameters (in)	<code>target_idx</code>	Index of the reference fruit to compare against.
Parameters (inout)	None	No inout parameters.
Parameters (out)	None	No output parameters.
Return value	<code>fruit_db_status_t</code>	Returns <code>FRUIT_DB_OK</code> if the reading matches the target fruit. Returns <code>FRUIT_DB_ERR_INVALID_INDEX</code> if the target index is invalid or empty. Returns <code>FRUIT_DB_ERR_NO_MATCH</code> if the color distance is larger than the tolerance.
Description	Normalizes the current RGB reading, computes its Euclidean distance from the selected reference fruit, and compares that distance against <code>DB_MATCH_TOLERANCE</code> .	
Constraints	Compares only against one selected target index. It does not automatically search for the closest matching fruit across the whole database. Negative RGB values are not rejected.	

Available via	fruit_db.h	
----------------------	------------	--

fruit_db_get_count

Function Name	fruit_db_get_count	
Syntax	uint8_t fruit_db_get_count(void)	
Parameters (in)	None	No input parameters.
Parameters (inout)	None	No inout parameters.
Parameters (out)	None	No output parameters.
Return value	uint8_t	Current number of valid fruit records. Expected range is 0 to DB_MAX_FRUITS.
Description	Returns the value of the internal total_frutas counter.	
Constraints	Count accuracy depends on proper initialization and correct maintenance by add/delete operations. Current repeated calls to fruit_db_init() may corrupt this count unless corrected.	
Available via	fruit_db.h	

fruit_db_get_table

Function Name	fruit_db_get_table	
Syntax	const fruta_registro_t* fruit_db_get_table(void)	
Parameters (in)	None	No input parameters.
Parameters (inout)	None	No inout parameters.
Parameters (out)	None	No output parameters.
Return value	const fruta_registro_t*	Pointer to the first element of the internal database table.
Description	Provides read-only access to the internal fruit database so menus or display modules can list fruit names and reference values.	
Constraints	The returned pointer references internal static storage. The caller must not cast away const and modify the table directly.	
Available via	fruit_db.h	

4.4. Internal Function Definitions

db_strcpy

Function Name	db_strcpy	
Syntax	static uint8_t db_strcpy(char *dest, const char *src)	
Parameters (in)	src	Source null-terminated string to copy.
Parameters (inout)	None	No inout parameters.
Parameters (out)	dest	Destination buffer where the copied string is written.
Return value	uint8_t	Number of visible characters copied, not including the null terminator.
Description	Copies characters from src into dest until the null terminator is reached or until DB_MAX_NAME_LEN - 1 characters have been copied. It then appends \0.	
Constraints	Does not validate whether src or dest is NULL. Assumes dest has at least DB_MAX_NAME_LEN bytes available. Internal linkage prevents use outside fruit_db.c.	

5. Future Enhancements

- Correct the return types in fruit_db.c so they match the declarations in fruit_db.h and use fruit_db_status_t instead of uint8_t for status-returning functions.
- Reset total_frutas = 0 inside fruit_db_init() before loading default fruit records.
- Add NULL pointer validation for nombre in fruit_db_add() and for src / dest in db_strcpy() if defensive programming is required.
- Add validation to reject negative RGB values because RGB readings should normally be non-negative.
- Add a function to automatically search all valid fruits and return the closest match instead of requiring a target_idx.
- Add an output parameter to fruit_db_compare() or a new function to return the computed distance for debugging and calibration.
- Add an API to update an existing fruit record without deleting and re-adding it.
- Store calibration metadata, such as sample count, timestamp, or sensor gain settings, if the application later requires traceability.

- Consider replacing exact float comparison `suma == 0.0f` with a small epsilon threshold if noisy floating-point input is expected.
-